

Depth-First Search (DFS) in Graphs

◆ Prompt

What is Depth-First Search (DFS) in a graph?

✓ Answer

- DFS is a **graph traversal algorithm** that explores vertices **as deep as possible along each branch** before backtracking.
- Analogy: A person exploring a maze with a **string and paint**:
 - Paint a vertex as visited.
 - Move along an edge to an unvisited vertex.
 - If stuck (dead end), **backtrack** along the path until you find a vertex with unexplored edges.
- DFS is useful for:
 - Checking **connectivity**
 - Finding **paths**
 - Detecting **cycles**
 - Topological sorting
 - Finding **strongly connected components** in directed graphs

◆ Prompt

Explain the DFS algorithm step by step.

✓ Answer

Start: Pick a starting vertex u

Mark u as visited (paint it)

Explore edges from u :

- For each outgoing edge (u, v) :
 - If v is unvisited, recurse DFS on v
 - If v is visited, ignore the edge

Backtracking:

- When all edges from u lead to visited vertices, return to the previous vertex

Termination:

- DFS ends when all reachable vertices from the start vertex are visited

DFS explores as far as possible along each path before exploring alternatives.

DFS Algorithm (Pseudo-Code)

DFS(G, u):

```
# Assume u is already marked as visited

for each outgoing edge  $e = (u, v)$  of  $u$ :

    if  $v$  is not visited:

        mark  $v$  as visited

        DFS( $G, v$ )
```

- Recursive implementation naturally handles **backtracking**
- Time complexity: $O(n + m)$ for adjacency list graphs

DFS Properties

◆ Prompt

What are the important properties of DFS?

✓ Answer

Visits all reachable vertices from the starting vertex u .

DFS tree:

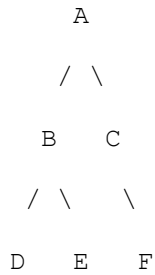
- Each edge used to discover a new vertex is a **tree edge**
- Other edges are **nontree edges**:
 - **Back edge**: connects a vertex to an ancestor
 - **Forward edge**: connects a vertex to a descendant
 - **Cross edge**: connects two unrelated vertices (neither ancestor nor descendant)

Applications:

- Detect cycles (back edges)
- Topological sorting in DAGs
- Strongly connected components in directed graphs
- Maze solving or path finding

Example DFS Traversal

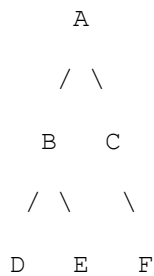
Graph:



DFS starting at A (preorder traversal):

1. Visit A → mark as visited
2. Go to B → mark as visited
3. Go to D → mark as visited (dead end, backtrack)
4. Go to E → mark as visited (dead end, backtrack to B)
5. Backtrack to A → go to C → mark as visited
6. Go to F → mark as visited (dead end)

DFS Tree:



- **Tree edges (solid lines):** edges used to discover vertices
- **Back edges (dashed lines):** edges to ancestors, if present

DFS vs BFS

Feature	DFS	BFS
Traversal order	Depth first	Level by level
Shortest path guarantee	✗ No	✓ Yes
Data structure	Stack (or recursion)	Queue
Connectivity check	✓ Yes	✓ Yes
Cycle detection	✓ Yes (back edges)	Limited
Use case	Topological sort, SCC	Shortest paths, social networks

Applications of DFS

- **Cycle detection** in graphs
- **Topological sorting** of DAGs
- **Finding connected components**
- **Maze solving / pathfinding**
- **Strongly connected components** in directed graphs

DFS Visualization (Step-by-Step)

Start at A:

Step 0: A

Step 1: B (go deep)

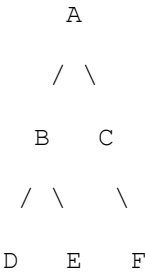
Step 2: D (dead end, backtrack)

Step 3: E (dead end, backtrack)

Step 4: C (go deep)

Step 5: F (dead end, backtrack)

DFS Tree:



Classifying Graph Edges Using DFS

◆ Prompt

How does DFS classify edges in a graph?

✓ Answer

- During **DFS**, edges are explored as the traversal progresses.
- The edges are classified into **two main types**:

Tree (discovery) edges:

- An edge (u, v) is a **tree edge** if it is used to **discover a new vertex v** .
- These edges form the **DFS tree**.

Nontree edges:

- An edge (u, v) is a **nontree edge** if v was **already visited**.
- These edges are classified differently based on the graph type:

◆ Prompt

What are the nontree edges in undirected graphs?

✓ Answer

- **Back edges only:**
 - All nontree edges connect the current vertex to an **ancestor** in the DFS tree.
 - There are no forward or cross edges in an undirected graph.

Summary Table (Undirected Graphs)

Edge Type	Definition
Tree/Discovery	Edge used to reach a previously unvisited vertex
Back edge	Edge connecting a vertex to an ancestor in DFS tree

Edge classification in directed graphs

◆ Prompt

What are the types of nontree edges in directed graphs?

✓ Answer

- In a **directed graph**, nontree edges can be one of **three types**:

Edge Type	Definition
Tree/Discovery	Edge used to reach a new (unvisited) vertex
Back edge	Edge connecting a vertex to an ancestor in the DFS tree
Forward edge	Edge connecting a vertex to a descendant in the DFS tree
Cross edge	Edge connecting a vertex to another vertex that is neither ancestor nor descendant

Example: Directed Graph DFS

Graph vertices: BOS, ORD, SFO, LAX, DFW, JFK, MIA

- Start DFS at BOS

DFS tree edges: Solid lines

Back edges: Dashed lines

Forward / cross edges: Dotted lines

Example classifications from Figure 14.8:

Edge	Type	Notes
(ORD, DFW)	Back edge	DFW is ancestor of ORD in DFS tree
(DFW, ORD)	Forward edge	ORD is descendant of DFW in DFS tree
(BOS, SFO)	Forward edge	SFO is descendant of BOS in DFS tree
(SFO, LAX)	Cross edge	LAX is neither ancestor nor descendant of SFO

Example: Undirected Graph DFS

Graph vertices: A, B, C, D, E, F, G, H

- Start DFS at A
- DFS explores edges **alphabetically**

DFS tree edges: Solid lines

Back edges: Dashed lines

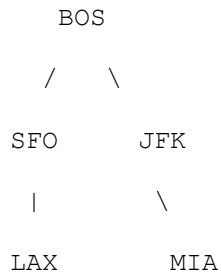
Steps from Figure 14.9:

1. Start at A → visit B → tree edge (A, B)

2. Explore B $\rightarrow$ visit C $\rightarrow$ tree edge (B, C)
3. Explore C $\rightarrow$ visit G $\rightarrow$ tree edge (C, G)
4. Edge (G, C) $\rightarrow$ back edge (already visited ancestor)
5. Continue DFS until all vertices visited

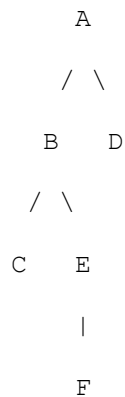
Visual Summary

Directed Graph Edge Classification



- Tree edges: solid
- Back edges: dashed
- Forward edges: dotted
- Cross edges: dotted

Undirected Graph Edge Classification



- Tree edges: solid
- Back edges: dashed (e.g., (G,C))